PHASE 6

Schemas, Routing and Pipeline

Core source code: schemas.py, routes.py and comic_service.py

COMICCRAFT • AI COMIC STORY CREATOR • TECHNICAL DOCUMENTATION

1. Core Application Source Code

The sections below reproduce the primary application modules in full, for reference and review.

1.1 Data Schemas — app/schemas.py

All request and response payloads are strongly typed with Pydantic v2, giving automatic validation, OpenAPI documentation and a uniform JSON error envelope.

Python — app/schemas.py (abridged)

```
"""Pydantic schemas used for request validation and API responses."""

from __future__ import annotations
from typing import List, Optional
from pydantic import BaseModel, Field, field_validator
from app.config import settings

ALLOWED_TONES = ["Funny", "Dramatic", "Adventure", "Dark", "Emotional",
                 "Light-hearted", "Epic", "Mysterious"]
ALLOWED_ART_STYLES = ["Anime", "Comic Book", "Pixel Art", "Cartoon",
                      "Realistic", "Manga", "Fantasy", "Cinematic"]


class PromptRequest(BaseModel):
    """Payload accepted by the JSON comic-generation endpoint."""

    story_prompt: str = Field(..., min_length=10, max_length=settings.MAX_PROMPT_LENGTH)
    character_name: str = Field(..., min_length=1, max_length=settings.MAX_NAME_LENGTH)
    setting: str = Field(..., min_length=2, max_length=settings.MAX_NAME_LENGTH)
    tone: str = Field(..., min_length=2, max_length=settings.MAX_NAME_LENGTH)
    art_style: str = Field(..., min_length=2, max_length=settings.MAX_NAME_LENGTH)

    @field_validator("story_prompt", "character_name", "setting", "tone", "art_style")
    @classmethod
    def _strip_whitespace(cls, value: str) -> str:
        cleaned = " ".join(str(value).split())
        if not cleaned:
            raise ValueError("This field cannot be empty.")
        return cleaned


class ComicPanel(BaseModel):
    """A fully assembled comic panel ready for rendering or export."""

    panel_number: int
    title: str
    scene_description: str
    caption: str
    narration: str
    dialogue: List[str] = Field(default_factory=list)
    image_prompt: str
    image_url: Optional[str] = None
    image_path: Optional[str] = None
    image_error: Optional[str] = None
```

```
class ComicResponse(BaseModel):
    """Successful response returned by the JSON generation endpoint."""

    success: bool = True
    title: str
    character_name: str
    setting: str
    tone: str
    art_style: str
    panels: List[ComicPanel]
    pdf_url: Optional[str] = None
    warnings: List[str] = Field(default_factory=list)


class ErrorResponse(BaseModel):
    """Uniform error envelope returned to API clients."""

    success: bool = False
    error: str
    code: str = "generation_error"


class HealthResponse(BaseModel):
    """Health-probe payload."""

    status: str
    app: str
    environment: str
    gemini_configured: bool
    image_provider: str
    image_configured: bool
```

1.2 Routing Layer — app/routes.py

Defines every HTTP endpoint: the landing page, the two comic-generation entry points (HTML form and JSON API), the download route (guarded against path traversal), and system utilities.

Python — app/routes.py (abridged)

```
"""All ComicCraft HTTP routes: pages, form flow, JSON API and utilities."""

router = APIRouter()


@router.get("/", response_class=HTMLResponse, name="home", tags=["Pages"])
async def home(request: Request) -> HTMLResponse:
    """Render the landing page and comic creation workspace."""
    return templates.TemplateResponse(request, "index.html", _form_context(request))


@router.get("/health", response_model=HealthResponse, tags=["System"])
async def health() -> HealthResponse:
    """Lightweight health and configuration probe."""
    return HealthResponse(
        status="ok",
        app=settings.APP_NAME,
        environment=settings.APP_ENV,
        gemini_configured=settings.gemini_configured,
        image_provider=settings.IMAGE_PROVIDER,
        image_configured=settings.image_configured,
    )


@router.post(
    "/generate-comic/json",
    response_model=ComicResponse,
    responses={422: {"model": ErrorResponse}, 502: {"model": ErrorResponse}},
    tags=["Comic"],
)
async def generate_comic_json(payload: PromptRequest) -> JSONResponse:
    """Generate a comic from a raw JSON payload and return structured data."""
    try:
        comic = create_comic(**payload.model_dump())
    except ComicGenerationError as exc:
        return JSONResponse(
            status_code=502,
            content=ErrorResponse(error=str(exc), code=exc.code).model_dump(),
        )
    except Exception:  # noqa: BLE001
        logger.exception("Unexpected failure in JSON generation")
        return JSONResponse(
            status_code=500,
            content=ErrorResponse(
                error="Something went wrong while creating your comic.",
                code="server_error",
            ).model_dump(),
        )
    return JSONResponse(content=ComicResponse(**comic).model_dump())


@router.get("/download", tags=["Comic"])
async def download_pdf(pdf_url: str, title: Optional[str] = None) -> FileResponse:
    """Serve an exported PDF as a download, guarding against path traversal."""
    relative = pdf_url.replace("/static/", "", 1).lstrip("/")
    candidate = settings.STATIC_DIR / relative
    try:
        safe_path = ensure_within(settings.EXPORTS_DIR, candidate)
    except ValueError:
        raise HTTPException(status_code=400, detail="Invalid download request.")

    if not Path(safe_path).exists():
        raise HTTPException(status_code=404, detail="That comic export is no longer available.")

    filename = f"{(title or 'comiccraft-comic').strip()[:60]}.pdf".replace("/", "-")
    return FileResponse(path=str(safe_path), media_type="application/pdf", filename=filename)
```

1.3 Pipeline Orchestration — app/services/comic_service.py

The heart of the application. create_comic() drives the outline → story → illustration → layout → export pipeline, converting every internal failure into a safe, user-facing ComicGenerationError.

Python — app/services/comic_service.py (abridged)

```
"""Orchestrates the full comic pipeline: outline -> story -> images -> layout -> PDF."""

class ComicGenerationError(RuntimeError):
    """User-facing pipeline failure with a safe, friendly message."""

    def __init__(self, message: str, code: str = "generation_error") -> None:
        super().__init__(message)
        self.code = code


def create_comic(
    story_prompt: str,
    character_name: str,
    setting: str,
    tone: str,
    art_style: str,
    export_pdf: bool = True,
) -> Dict[str, Any]:
    """Run the end-to-end comic generation pipeline."""
    if not settings.gemini_configured:
        raise ComicGenerationError(
            "ComicCraft is not configured yet. Add your "
            "GEMINI_API_KEY to the .env file and restart the server.",
            code="missing_api_key",
        )

    warnings: list[str] = []

    # --- Stage 1: outline ------------------------------------------------
    outline_result = generate_outline(
        story_prompt, character_name, setting, tone, art_style
    )
    title = outline_result["title"]
    panels = outline_result["panels"]
    appearance = outline_result.get("character_appearance", "")

    # --- Stage 2: story --------------------------------------------------
    story = generate_story(
        title, panels, character_name, setting, tone, art_style, appearance
    )

    # --- Stage 3: illustrations ------------------------------------------
    image_results, image_warnings = generate_panel_images(
        panels, slug=sanitize_filename(title), art_style=art_style
    )
    warnings.extend(image_warnings)

    # --- Stage 4: layout -------------------------------------------------
    layout = build_comic_layout(panels, story, image_results)

    # --- Stage 5: PDF export ---------------------------------------------
    pdf_url = None
    if export_pdf:
        pdf_path = save_pdf(
            layout,
            title=title,
            metadata={"character_name": character_name, "setting": setting,
                      "tone": tone, "art_style": art_style},
        )
        pdf_url = static_url(pdf_path)

    return {
        "success": True,
        "title": title,
        "character_name": character_name,
        "setting": setting,
        "tone": tone,
        "art_style": art_style,
        "panels": layout,
        "pdf_url": pdf_url,
        "warnings": warnings,
    }
```